La capa de aplicación.

La Web y HTTP (*1ª Parte*).

Tema 2: La capa de aplicación

2.1 Principios de las aplicaciones de red.

2.2 La Web y HTTP *(1ª parte).*

2.3 Correo electrónico en Internet.

2.4 DNS: el servicio de directorio de Internet *(2ª parte).*

2.5 Distribución de archivos P2P.

2.6 Flujos de vídeo y redes de distribución de contenidos.

2.7 Programación de sockets: creación de aplicaciones de red.

Introducción a la Web

WWW: World Wide Web → La Web

- Servicio para acceso a información, a recursos vinculados y distribuidos en muchos hosts.
- Permite:
  - Recibir y publicar información
  - "Navegar" por un "océano" de información a través de hiperenlaces, y la interacción de diferentes páginas Web.
- Funciona:
  - Bajo demanda.
    - Usuarios reciben y publican "lo que quieren" y "cuando quieren"
  - Mediante el modelo Cliente – Servidor.

Introducción a la Web

- Elementos de la arquitectura Web
  - Páginas.
    - HTML, hipertexto, hiperenlaces.
  - Objetos
    - Pueden residir o no en el mismo host.
  - Direccionado
    - URL: Uniform Resource Locator.
  - Entidades funcionales.
    - Cliente Web (navegador).
    - Servidor Web.
  - Protocolo HTTP.

Introducción a la Web

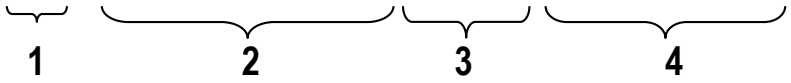
- Páginas
  - Contienen “objetos”
  - Resultado que muestra el navegador tras seguir instrucciones del fichero HTML.
  - Generalmente formada por:
    - Fichero HTML base.
      - Referencia a los otros objetos de la página mediante sus correspondientes URL.
    - Otros objetos referenciados en diferentes formatos.
      - Texto ordinario, hipertexto, imágenes, vídeos, audio …
      - Vinculados o no a otras páginas.
- Objetos
  - Archivos:
    - HTML, JPEG, etc.
    - Cada uno direccionable a través de un URL.

Direccionado

- En base a un URL: **U**niform **R**esource **L**ocator.
  - Elemento para el direccionamiento de recursos.
- Parámetros del URL:
  1. Identificador de servicio (protocolo).
  2. Nombre de dominio del host servidor.
  3. Ruta de acceso al objeto.
  4. Nombre del objeto.

Hay definidos otros tipos de URL, según tipo servicio (protocolo de acceso)

- Sintaxis URL.
  - Caracteres "://" separan primer parámetro del segundo.
  - Caracter "/" separa resto de parámetros entre sí.
- Ejemplo:
  - http://www.escuela.es/dptotic/imagen1.jpg



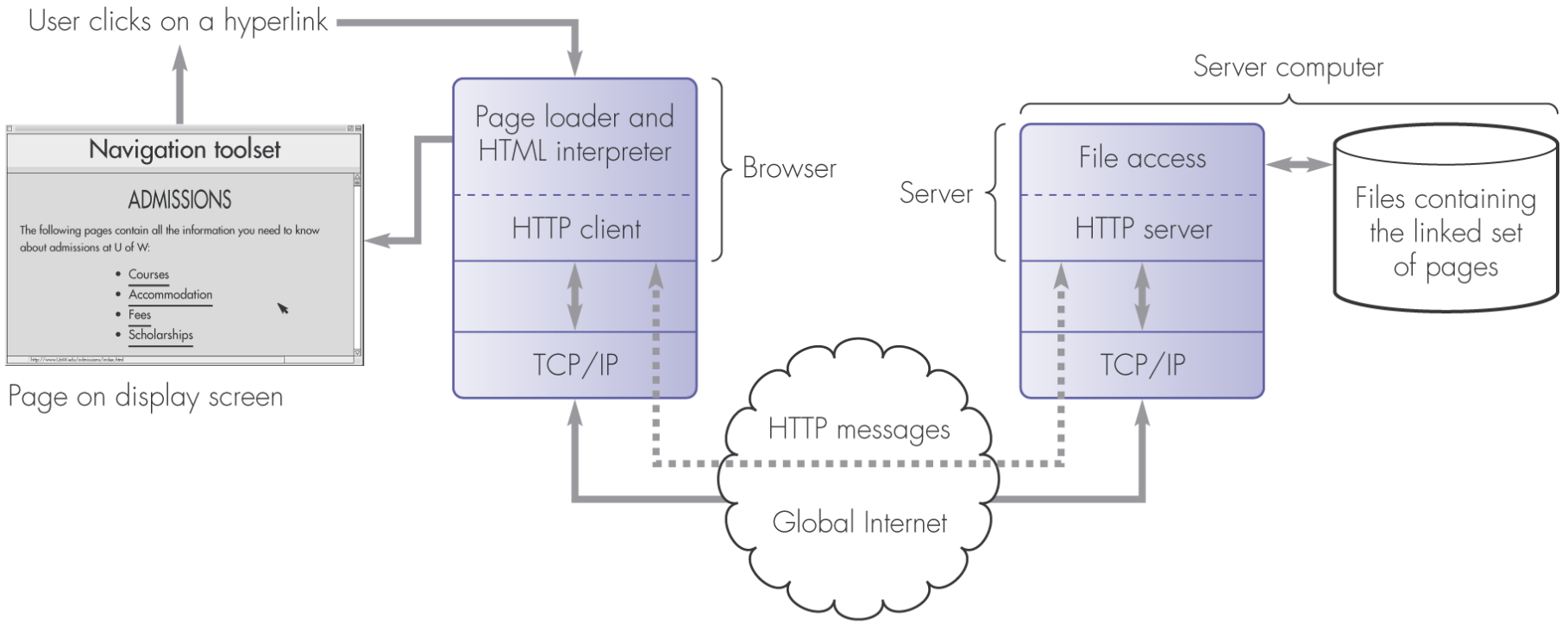
Introducción a la Web

- Cliente Web (navegador)
  - Comprende:
    - Agente de Usuario.
      - Software que actúa como interfaz entre usuario y aplicación Web.
      - Permite “atrapar” y visualizar páginas, y “navegar” por sus contenidos.
      - Presenta diferentes características de configuración.
    - Cliente HTTP.
      - Software de comunicación que implementa la parte cliente HTTP.
  - Navegadores más conocidos:
    - Internet Explorer 10.
    - Firefox.
    - Chrome, y otros más…

- Servidor Web
  - Comprende tres entidades funcionales:
    - Software de acceso a ficheros.
    - Base de datos.
      - Alberga objetos.
      - Cada objeto direccionable con el correspondiente URL.
    - Servidor HTTP.
      - Software de comunicación que implementa la parte servidora HTTP.
  - Servidores Web más conocidos:
    - Apache.
    - MS Internet Information Server.

Introducción a la Web

Elementos de la arquitectura Web



Note: Hyperlinks contain a URL which includes the domain name of the server computer and the name of the file containing the HTML code of the selected page

Fuente: Halsall, *Computing Networking and the Internet,* 5th Edition © Pearson Education Limited 2005

HTTP

- HTTP: **H**yper **T**ext **T**ransfer **P**rotocol
  - Protocolo para la transferencia de recursos.
  - Recursos: archivos y resultado, p.e, de la:
    - Consulta a una base de datos.
    - Ejecución de un programa.
    - Traducción automática de un documento.
  - Especificado en dos versiones compatibles entre si:
    - HTTPv1.0, especificado en la RFC 1945.
    - HTTPv1.1, especificado en la RFC 2616.
  - Opera en base al modelo Cliente – Servidor.

HTTP

- HTTP: **H**yper **T**ext **T**ransfer **P**rotocol
  - Basado en simples “transacciones”: petición – respuesta.*
    - Cliente solicita objetos, genera “peticiones”.
      - Mensajes ASCII (formato entendible por humanos).
    - Servidor transfiere objetos, genera “respuestas”.
      - Mensajes ASCII tipo MIME (RFC 822).
    - Es básicamente un protocolo de “demanda”.
  - Protocolo SIN estado y NO conectivo.
    - El servidor no guarda información de clientes ni de transacciones.
    - Simplifica el diseño de servidores → Servidores Web de gran rendimiento, pueden manejar muchas conexiones TCP y las correspondientes transacciones HTTP.
  - Protocolo no fiable y no seguro.

Los protocolos con estado son complejos

- Requieren mantener la “historia” pasada (el estado).
- Si servidor/cliente falla, sus puntos de vista del estado pueden ser inconsistentes

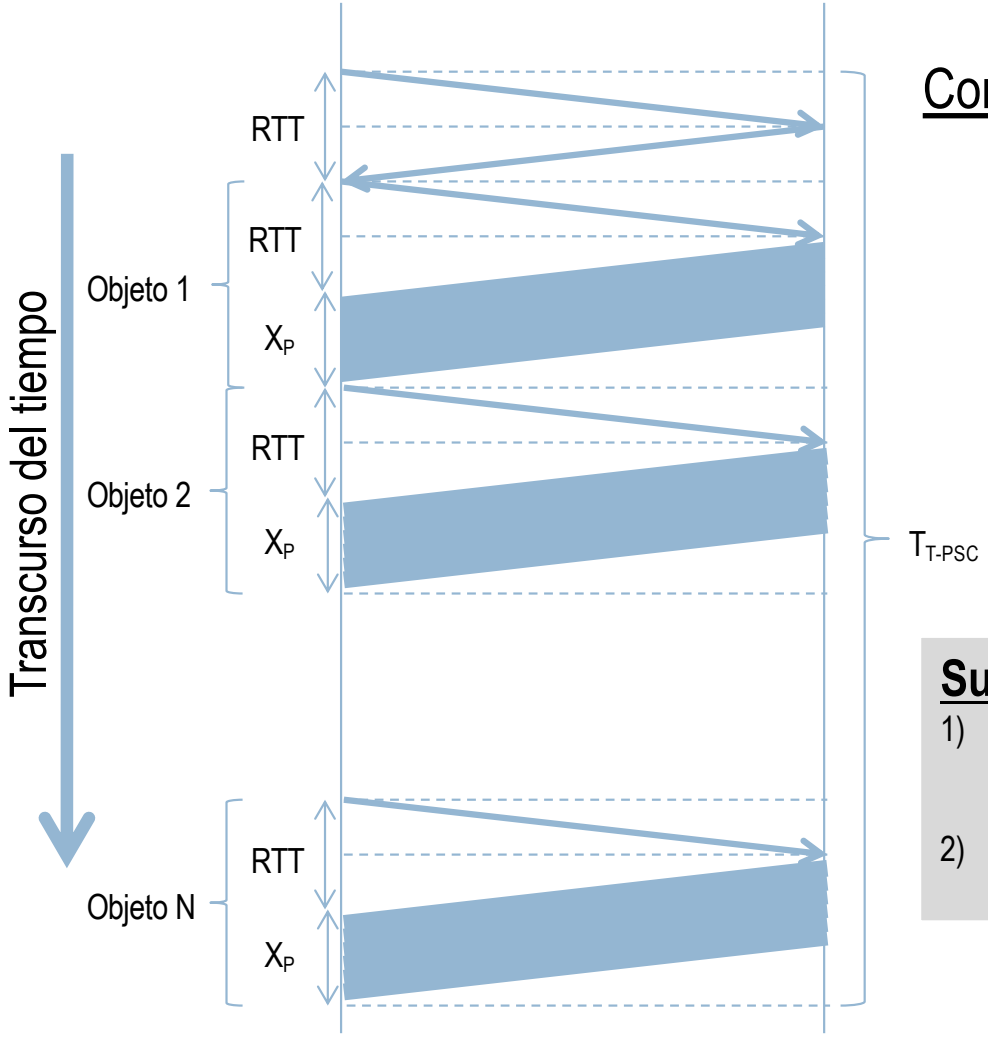
HTTP

- Protocolo de Transporte
  - El estándar NO especifica. En la práctica TCP → HTTP no requiere procedimientos de control de errores.
  - Toda transacción HTTP (solicitud –respuesta) requiere conexión TCP.
- Procedimiento
  - Cliente solicita conexión TCP al puerto 80 del servidor.
  - Servidor acepta conexión del cliente (se crea “socket de conexión” en el servidor).
  - Navegador y servidor intercambian mensajes HTTP.
  - Se cierra la conexión TCP.
- Tipos de conexiones TCP para sesiones HTTP
  - No persistentes.
    - Solución única para HTTPv1.0. Soportada también por HTTPv1.1.
  - Persistentes.
    - Solución por defecto para HTTPv1.1.

Conexión persistente

- Por una misma conexión TCP:
  - Discurren todas las transacciones HTTP.
  - Se transfieren todos los objetos de una página si están todos en el mismo servidor.
  - Transcurrido un tiempo (configurable) sin transacciones el servidor cierra la conexión TCP.

- Modos.
  - Sin canalización (sin “pipelining”).
    - Sólo una transacción HTTP a la vez.
  - Con canalización (con “pipelining”).
    - Admite varias transacciones HTTP a la vez → Procesamiento en cadena.
    - Solución por defecto para HTTPv1.1



Conexión TCP persistente SIN CANALIZACIÓN

Para "N" objetos a descargar, todos de igual tamaño, el tiempo total de descarga es:

$$T_{T\text{-}PSC} = RTT + N.(RTT + X_P)$$

Supuestos:

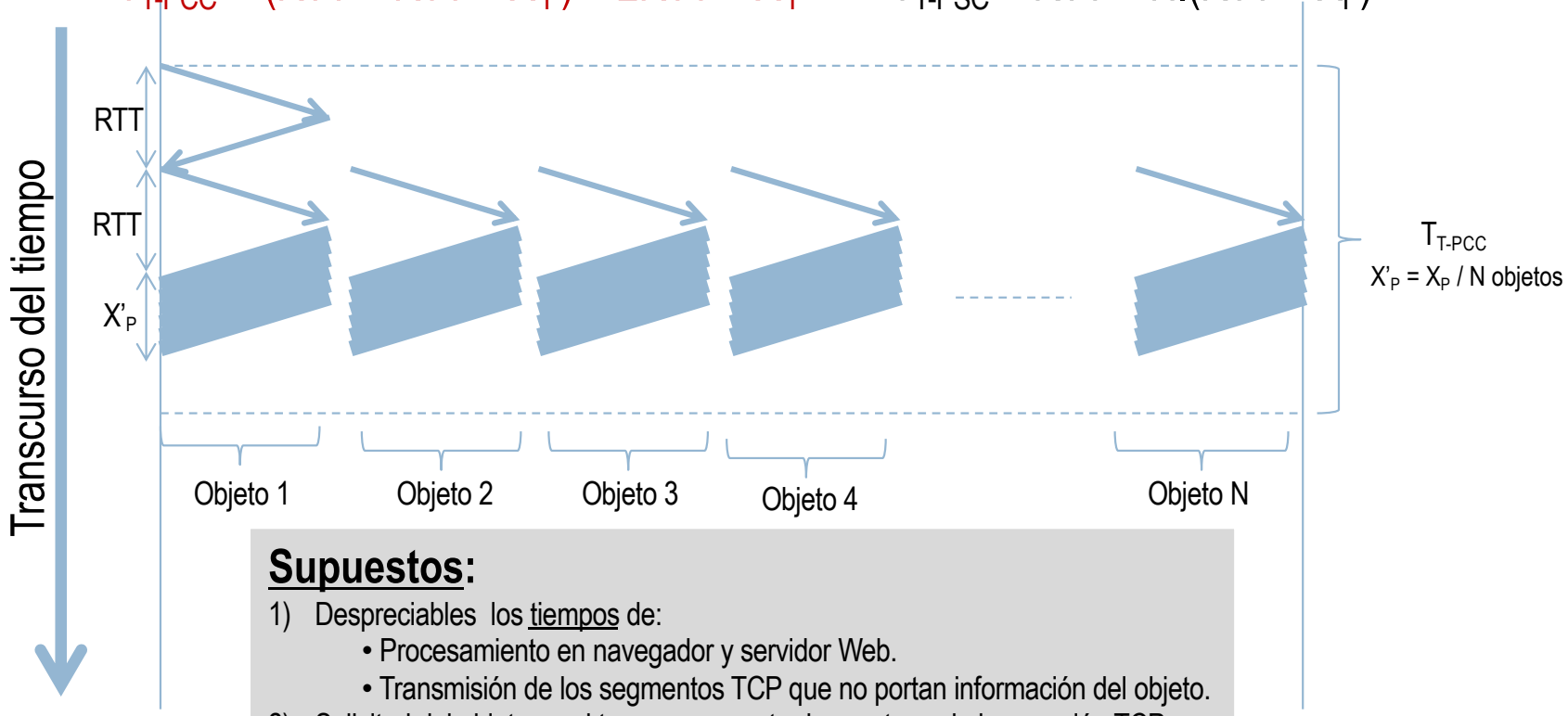
1) Despreciables los tiempos de:
   - Procesamiento en navegador y servidor Web.
   - Transmisión de los segmentos TCP que no portan información del objeto.
2) Solicitud del objeto en el tercer segmento de apertura de la conexión TCP.
   - No siempre es así.

Lo representado en la figura acontece después que se ha descargado el fichero HTML base.

Conexión TCP persistente CON CANALIZACIÓN

El tiempo total de descarga para "N" objetos a descargar, todos de igual tamaño y un grado de canalización "N", es:

$$T_{T\text{-}PCC} \approx (RTT + RTT + X'_P) = 2RTT + X'_P \ll T_{T\text{-}PSC} = RTT + N.(RTT + X_P)$$



Supuestos:

1) Despreciables los tiempos de:
   - Procesamiento en navegador y servidor Web.
   - Transmisión de los segmentos TCP que no portan información del objeto.
2) Solicitud del objeto en el tercer segmento de apertura de la conexión TCP.
   - No siempre es así.

Lo representado en la figura acontece después que se ha descargado el fichero HTML base, y conlleva a compartir "simultáneamente" el ancho de banda entre los "N" objetos.

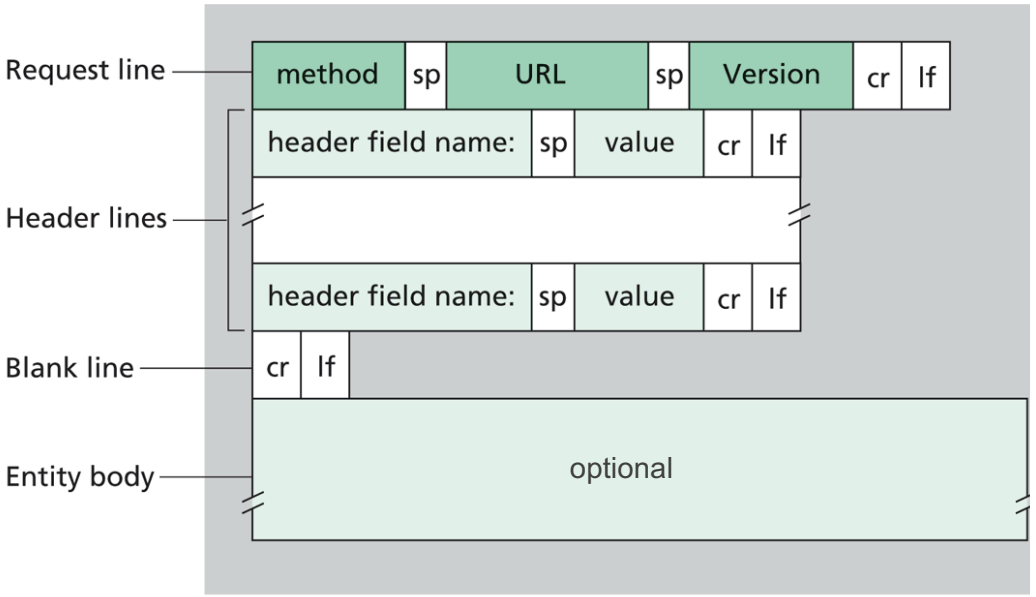
HTTP

- Tipos de mensajes
  - Métodos.
    - Solicitudes tipo ASCII.
  - Códigos de estado.
    - Respuestas tipo ASCII MIME.

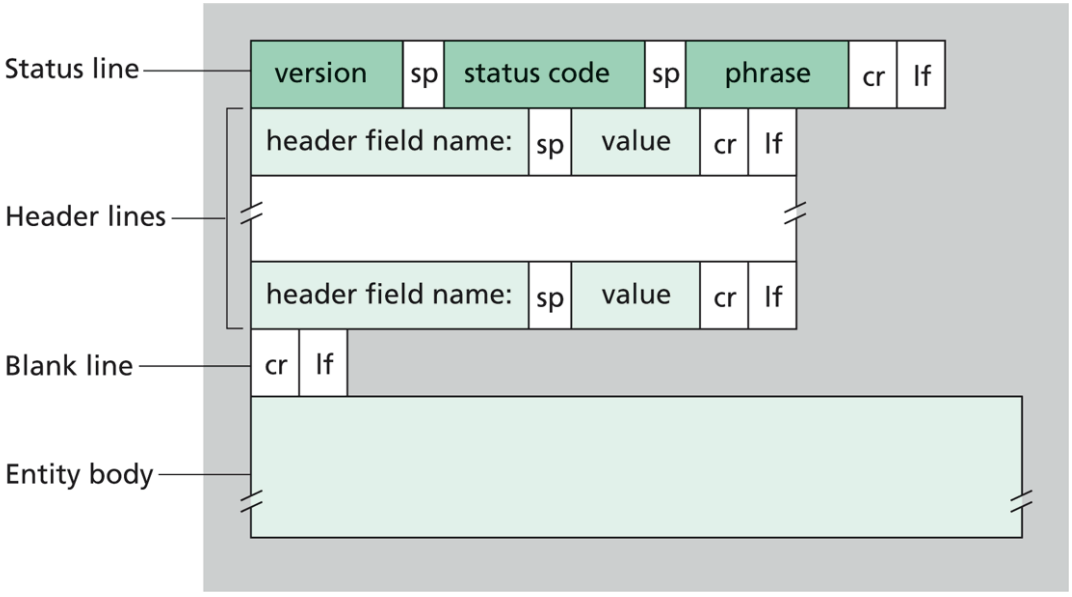
- Formato de mensajes.

Línea de inicio	← de solicitud o de respuesta.
Cabeceras	← de solicitud o de respuesta.
Línea en blanco	← como separador.
Cuerpo	← opcional.

Formato general de mensajes HTTP de “solicitud” y de “respuesta”



General format of a request message



General format of a response message

Más sobre mensajes HTTP

- Cabeceras de solicitud.
  - Dan información del navegador y eventualmente del usuario.
- Cabeceras de respuesta.
  - Dan información del servidor y del “recurso”.
- Cuerpo.
  - En mensajes de “solicitud”:
    - Vacío, o
    - Parámetros relativos al “recurso” solicitado.
  - En mensajes de “respuesta”:
    - “Recurso” solicitado o
    - Texto relativo a un error.

HTTP

- Métodos (solicitudes) más significativos
  - **GET**: para descargar página/objeto (obtener información)*.
    - Mensaje de solicitud más utilizado.
    - Si es el caso, envío de parámetros conjuntamente con el URL.
  - **POST**: para descargar objeto (enviar información).*
    - Envío de parámetros/información en el cuerpo del mensaje. P.e, para completar un formulario.
    - Lo que devuelve el servidor depende del contenido enviado.
  - **HEAD**: similar a GET, pero para descargar sólo el encabezado de la página/objeto.
    - Para supervisión y depuración. P.e, última modificación de un objeto, comprobar enlaces, etc.
  - **PUT**: para cargar nueva página/objeto.
  - **DELETE**: para eliminar página/objeto.
  - **OPTIONS**: para consultar opciones.
    - P.e, propiedades del servidor o de algún recurso.

- Códigos de estado (respuestas)
  - Código de tres caracteres numéricos.
  - Primer carácter indica **tipo** de código de estado.
  - Tipos:

    1xx: información, respuesta provisional (100–199).
    2xx: éxito en la resolución de la solicitud (200-299).
    3xx: redirigir solicitud, necesarias más acciones (300-399).
    4xx: error del cliente (400-499).
    5xx: error del servidor (500-599).

  - Ejemplos en el libro de texto.
  - Más información en la RFC 2616.

Cabeceras

¿Qué cabeceras se utilizan en los mensajes HTTP? Depende:

- Por parte del navegador, en los métodos:
  - Tipo de mensaje.
  - Versión del protocolo.
  - Configuración hecha por el usuario (p.e, idioma preferido).
  - Tipo y versión del navegador.
  - Disponibilidad en caché del objeto.
- Por parte del servidor, en los códigos de estado:
  - Tipo de mensaje.
  - Versión del protocolo.
  - Configuración del servidor.
  - Información relativa al recurso (si es el acaso).
  - Cómo tratar el recurso (caché, no caché, cookie, no cookie, etc.).

Algunas cabeceras comunes de mensajes de SOLICITUD/RESPUESTA

Cabecera		Contenido
Date		Fecha y hora en que se envía el mensaje
Connection		Indica en relación con la conexión TCP, cerrar o mantener

Algunas cabeceras de mensajes de SOLICITUD

Cabecera		Contenido
User-Agent		Información del Navegador y su plataforma
Accept		Tipos de contenidos que el Cliente puede aceptar
Accept-Charset		Conjunto de caracteres que el Cliente puede aceptar
Accept-Encoding		Codificaciones que el Cliente puede aceptar
Accept-Language		Idiomas que el Cliente puede manejar
Host		Nombre de dominio del Servidor
Authorization		Credenciales del Cliente (nombre de usuario y clave)
Cookie		Notificación de “cookie”, previamente establecida por un Servidor

Algunas cabeceras de mensajes de RESPUESTA

Cabecera		Contenido
Server		Información del Servidor
Content-Encoding*		Tipo de codificación del contenido
Content-Language		Idioma utilizado por el público al que va destinado el contenido
Content-Length*		Nº de octetos del contenido
Content-Type*		Tipo MIME del contenido
Last-Modified		Fecha y hora de la última modificación del recurso
Location		Indica al Cliente dónde enviar solicitud para el recurso solicitado
Set-Cookie		Cookie que el Servidor envía al Cliente
Authenticate		Indicación al Cliente que se requiere autenticación

*: también pueden estar en cabeceras de mensajes de solicitud.

HTTP

- Ejemplo: cliente solicita el objeto “doc.html”.

```
GET /doc1.html HTTP/1.0
Accept: text/html
Accept: image/gif
User-Agent:  Lynx/2.2  libwww/2.14
From: pepe@info.ula.es
/* esto es una línea en blanco */
```

- Método GET, fichero que se solicita y versión HTTP.
- Lista de tipos MIME que puede aceptar.
- Identificación del SW de la máquina cliente (normalmente SO y navegador).
  - Permite al servidor optimizar ficheros para tipo particular de navegador.
- Dirección de correo electrónico del usuario.
- Línea en blanco, final de cabeceras HTTP.

- Ejemplo: servidor responde.

```
HTTP/1.0  200 OK
Date: Friday, 23-Feb-01 16:30:00 GMT
Server: Apache/1.1.1
Content-type: text/html
Content-length: 230
/* esto es una linea en blanco */
<HTML><HEAD><TITLE> ..... </HTML>
```

- Versión 1.0 de HTTP.
- Código de estado 200 → Solicitud procesada satisfactoriamente.
- Fecha y hora en la que se envía el mensaje.
- Tipo de servidor: Apache.
- Tipo de contenido: texto en formato HTML.
- Longitud en octetos del contenido: 230.
- Línea en blanco, final de cabeceras HTTP.
- Cuerpo del mensaje con el recurso.